

Persona (DBS) / JD (ERS)	Module Name	Sub Topic	SME Hours	Hands-on Hours	E-Learning Hours
Java Fullstack	1. Core Java Fundamentals				
		Java Platform Basics: JVM, JRE, JDK; JVM architecture; compilation & execution flow	1		
		Language Fundamentals: Data types (primitives, arrays); variables & constants; operators; type casting	1	2	
		Control Flow : if/else, switch; while, for, enhanced for; break, continue	1	2	
		OOP Concepts :Classes, fields, methods, constructors; this, super; access modifiers; packages	1	3	
		Inheritance & Polymorphism : Interfaces, abstract classes; method overloading & overriding; inheritance hierarchies	2	3	
		Exception Handling : try-catch-finally; throws vs throw; custom exceptions; checked vs unchecked	2	3	
		Collections Framework : List, Set, Map, Queue; Comparable & Comparator; iteration patterns; when to use HashMap vs TreeMap, ArrayList vs LinkedList	3	4	
		Generics : Generics basics; bounded types; wildcards; try-with-resources	1	2	
		Memory Management : Heap vs stack; garbage collection basics; common memory pitfalls	1		
		AI Integration : Using Copilot for boilerplate generation (POJOs, collections code); critically reviewing AI-generated OOP code	1	2	
		Assessment 1: Core Java Checkpoint Coding exercise: Fundamentals, OOP design, exception handling, collections	1		
	2. Modern Java (8–21)				
		Functional Programming : Lambda expressions; functional interfaces (Predicate, Function, Consumer, Supplier); method references	2	2	
		Streams API : Stream creation; intermediate & terminal operations; Collectors API; parallel streams awareness	2	2	
		Optional & Date/Time : Optional class patterns (avoid null); java.time API	1	2	
		Concurrency Essentials : Callable & Future; CompletableFuture basics; when to use async	1	2	
		Java 9–21 Key Features : Records; sealed classes; text blocks; switch expressions; var keyword; pattern matching	1	2	
		Immutable Collections & Modern APIs : List.of(), Map.of(); new String methods	1	2	
		AI Integration : Generating stream pipelines with Copilot; validating AI-generated functional code for correctness and readability	1	2	
	3. Version Control – Git				
		Git Fundamentals : Init, clone, add, commit, push, pull; .gitignore; SSH setup	1	2	1.25
		Branching & Merging : Branches, merge, rebase; resolving conflicts	1	2	
	4. Build Tools, IDE & Debugging				
		Maven : Project structure; POM; dependencies; lifecycle; profiles	1	2	
		IDE & Debugging : Breakpoints, conditional breakpoints, logpoints; step in/out/over; watch variables; call stacks; hot code replace	1	1	
		AI Integration : Using Copilot for debugging assistance; interpreting stack traces with AI	1	1	
	5. SQL & Database				
		Relational Database Concepts : Tables, relationships, normalization (1NF–3NF); when to use RDBMS	2		6.37
		CRUD Operations : CREATE, INSERT, UPDATE, DELETE; SELECT with WHERE, IN, BETWEEN, LIKE	2	4	
		Joins & Aggregations : INNER, LEFT, RIGHT, FULL joins; GROUP BY, HAVING; COUNT, SUM, AVG	2	2	
		Views, Indexing & Constraints : Index types; Views; foreign keys, unique, check constraints; cascading	2	2	
		Transactions : COMMIT, ROLLBACK; ACID properties; isolation levels awareness	1	2	
		AI Integration : Generating SQL queries with Copilot; validating AI-generated joins and aggregations for correctness	1	1	
		Assessment 2: SQL Checkpoint : Exercises on SQL	1		

Persona (DBS) / JD (ERS)	Module Name	Sub Topic	SME Hours	Hands-on Hours	E-Learning Hours
	6. Spring Boot & REST API Development				
		Spring Core Concepts : IoC, dependency injection; beans, scopes, lifecycle; annotations vs configuration	2	3	10.37
		Spring Boot Fundamentals : Auto-configuration; starters; project structure; application.properties / YAML; profiles	2	3	
		REST Design Principles : HTTP methods semantics (GET, POST, PUT, PATCH, DELETE); status codes (2xx, 4xx, 5xx); resource naming conventions; idempotency; statelessness	2	2	
		RESTful API Development : Controllers, request mapping; request/response DTOs; path variables, query params; content negotiation	2	2	
		Validation & Error Handling : Bean validation (@Valid, constraints); global exception handling (@ExceptionHandler); custom error responses; problem details	1	2	
		API Documentation : OpenAPI 3.0 / Swagger — generating, reading, and validating API specs	1	2	
		Spring Data JPA : Repositories; query methods; pagination & sorting; entity relationships; @Transactional; schema generation	3	4	
		AI Integration : Generating controllers, DTOs, and repository methods with Copilot; reviewing AI output for REST design violations and missing validations	1	2	
	7. Spring Security				
		Authentication & Authorization : Concepts; Spring Security filter chain; security configuration and implementation	2	2	3.4
		JWT Authentication : Token structure; login flow; token validation; stateless auth; refresh tokens	2	3	
		AI Integration : Reviewing Copilot-generated security configurations; exercises identifying security flaws in AI-generated auth code	1	2	
	8. Backend Testing & Code Quality				
		JUnit 5 : Test structure; assertions; lifecycle annotations; parameterized tests	1	2	3.35
		Mockito : Mocks, stubbing; behavior verification; argument captors	2	2	5.44
		Integration Testing : @SpringBootTest; Testcontainers basics; MockMvc for API testing	2	2	
		Best Coding Practices : Clean code fundamentals; separation of concerns; meaningful naming; avoiding magic values; defensive coding basics; writing	1	2	
		Coding Standards : Java coding conventions; naming standards; project and folder structure conventions; consistent exception handling patterns; logging practices	1	2	
		Code Coverage : Concepts; JaCoCo setup and reports; interpreting coverage metrics; coverage vs quality	2	2	
		SonarQube : Overview; scanning a project; reading results; fixing top issue types (bugs, smells, vulnerabilities)	1	2	
		AI Integration : Generating test cases with Copilot; critically reviewing AI-generated mocks for correctness	1	2	
		Assessment 3: Backend Checkpoint : Build a Spring Boot REST API with JWT auth, validation, JPA persistence	1		
	9. Logging & Monitoring				
		Structured Logging : SLF4J + Logback; log levels; log appenders; what to log, what NOT to log (no PII, no secrets)	1	2	
		Monitoring Awareness : Health endpoints; Spring Boot Actuator basics; what metrics matter in production	1	2	
	10. Design Patterns & SOLID				
		SOLID Principles : Single Responsibility; Open/Closed; Liskov Substitution; Interface Segregation; Dependency Inversion — with Java/Spring examples	2	3	1
		Key Design Patterns : Repository; Factory; Singleton; Adapter; Strategy; Builder — when to use each, with practical Spring examples	2	3	
	11. HTML5 & CSS3 Fundamentals				
		HTML5 : Semantic elements; forms & validation; non-semantic elements	3	3	4
		CSS3 : Box model; flexbox; grid; responsive design	3	3	
		Bootstrap5 : Grid system and responsive layout; containers, rows, and columns; typography and utility classes; common UI components (buttons, cards, navbars, forms); responsive utilities and breakpoints; basic theming and customization	3	3	

Persona (DBS) / JD (ERS)	Module Name	Sub Topic	SME Hours	Hands-on Hours	E-Learning Hours
	12. JavaScript Fundamentals				
		Language Basics : Syntax; variables (let, const, var); data types; operators; type coercion	1	2	22
		Functions & Scope : Function declarations/expressions; closures; hoisting; scope chain	2	2	
		DOM Manipulation & Events : DOM traversal; event handling; event delegation; form handling	1	2	
		ES6+ Features : Arrow functions; destructuring; spread/rest; template literals; modules (import/export)	2	2	
		Asynchronous JavaScript : Callbacks; Promises; async/await; Fetch API; error handling in async code	1	2	
		JSON & Data Handling : JSON parsing; working with APIs; data transformation patterns	1	2	
		Package Management : npm/yarn; package.json; dependency management; scripts	1	1	
		AI Integration : Using Copilot for JS code generation; identifying AI mistakes in async code and closures	1	1	
	13. TypeScript				
		TypeScript Fundamentals : Types, interfaces, enums; type inference; union & intersection types	1	2	3.45
		TypeScript with React : Typing components, props, state, events; generics in components	2	2	
		Advanced Patterns : Utility types; type guards; discriminated unions; module declarations	2	2	
	14. React.js				
		React Fundamentals : JSX; functional components (class components awareness); props; rendering lists & conditionals	2	3	6
		State & Events : useState; event handling; controlled components; forms (manual first)	2	3	
		Component Lifecycle & Hooks : useEffect; cleanup; useRef; useMemo; useCallback; custom hooks	2	3	
		Routing : React Router v6; nested routes; route params; protected routes; navigation	2	3	
		State Management : Context API; Redux Toolkit; when to use which (decision criteria)	2	3	
		API Integration : Axios and Fetch; loading/error states	2	3	
		Form Handling : React Hook Form; validation (Yup/Zod); multi-step forms	2	3	
		Advanced Topics : Error boundaries; lazy loading & Suspense	2	3	
		AI Integration : Generating components, hooks, and forms with Copilot; reviewing AI output for state management	1	2	
	15. Frontend Testing				
		Jest Basics : Test structure; matchers; mocking modules	2	2	
		React Testing Library : Rendering components; querying DOM; simulating events; testing hooks	2	2	
		E2E Awareness : Playwright — what it is, when to use, basic demo	2	2	
		Assessment 4 : Frontend Checkpoint : Build a React app consuming a provided REST API with routing, state management, form handling, and component tests	1		
	16. Full-Stack Integration				
		Connecting React to Spring Boot : CORS configuration; proxy setup; environment-based API URLs; .env management	1	2	
		End-to-End Data Flow : Form → API call → Controller → Service → DB → Response → UI update	1	2	
		Error Handling Across Stack : Backend validation errors → frontend display; global error handling patterns	1	2	
		Authentication Flow : JWT login → token storage → protected routes (frontend) → secured endpoints (backend)	1	2	
		AI Integration : Using Copilot to wire full-stack features end-to-end; debugging integration issues with AI	1	2	
	17. Microservices Concepts				

[illegible]